

Universidad de Costa Rica

Postgrado en Computación e Informática

Maestría Profesional en Computación e Informática

Laboratorio de Agentes Virtuales Inteligentes

PF - 3312

Proyecto investigativo #2

Benchmarking de modelos: LLM, TTS y STT.

Estudiante:

Dean Josué Vargas Delgado - B98194

I Ciclo, 2026



Introducción

Los avances recientes en inteligencia artificial generativa han impulsado el desarrollo de agentes virtuales capaces de comprender lenguaje natural, interpretar entradas multimodales y generar respuestas conversacionales de alta calidad. En particular, los Modelos de Lenguaje de Gran Escala (LLM), los sistemas de reconocimiento automático del habla (Speech-to-Text, STT) y los sistemas de síntesis de voz neuronal (Text-to-Speech, TTS) se han convertido en componentes fundamentales para la construcción de interfaces conversacionales modernas (Brown et al., 2020; OpenAI, 2023).

La aparición de arquitecturas multimodales como Gemini ha ampliado las capacidades tradicionales de los agentes virtuales, permitiendo integrar texto, audio, imágenes y otros tipos de información dentro de un mismo flujo de procesamiento (Google DeepMind, 2024). Paralelamente, el surgimiento de plataformas especializadas de inferencia acelerada, como Groq, y el crecimiento de modelos abiertos ejecutables localmente mediante herramientas como Ollama, han incrementado significativamente las alternativas disponibles para desarrolladores e investigadores.

Sin embargo, la selección de una arquitectura cognitiva adecuada no puede fundamentarse únicamente en criterios de popularidad o disponibilidad comercial. Aspectos como la latencia, precisión, escalabilidad, privacidad, facilidad de integración y costos operativos deben ser evaluados de manera sistemática mediante metodologías reproducibles de benchmarking. Estudios recientes han demostrado que el desempeño de modelos de inteligencia artificial puede variar considerablemente dependiendo del contexto de uso, los recursos computacionales disponibles y las características específicas de las tareas evaluadas (Liang et al., 2022).

En aplicaciones interactivas de tiempo real, como agentes virtuales integrados con motores gráficos y entornos inmersivos desarrollados en Unity, estas diferencias adquieren una relevancia crítica debido a que impactan directamente la experiencia del usuario final. Por esta razón, el presente trabajo desarrolla un benchmarking comparativo de tecnologías LLM, STT y TTS con el propósito de identificar las alternativas más adecuadas para distintos escenarios de implementación.

Objetivos

Objetivo General

Realizar un benchmarking comparativo de servicios de inteligencia artificial para agentes virtuales, evaluando modelos de lenguaje, reconocimiento de voz y síntesis de voz mediante criterios de desempeño, costo, privacidad, flexibilidad e integración.

Objetivos Específicos

1. Medir experimentalmente la latencia de distintos servicios LLM, STT y TTS utilizando un conjunto controlado de pruebas.
2. Comparar la calidad de las respuestas generadas por modelos de lenguaje, la precisión de transcripción de sistemas STT y la naturalidad de sistemas TTS.
3. Analizar los costos operativos y requerimientos de infraestructura asociados a cada alternativa evaluada.
4. Evaluar las implicaciones de privacidad y gobernanza de datos presentes en servicios cloud y soluciones de ejecución local.
5. Determinar el nivel de flexibilidad y personalización ofrecido por cada tecnología para su integración dentro de agentes virtuales.
6. Identificar las alternativas más adecuadas para distintos escenarios de implementación considerando restricciones de presupuesto, privacidad y requerimientos de tiempo real.

Metodología de Pruebas

Diseño del Banco de Pruebas

Con el objetivo de garantizar la reproducibilidad de los experimentos, se desarrolló un banco de pruebas modular en Python capaz de evaluar de forma independiente servicios de lenguaje natural (LLM), reconocimiento de voz (STT) y síntesis de voz (TTS). El sistema fue diseñado siguiendo principios de desacoplamiento entre la capa de benchmarking y la capa de aplicación, evitando que factores externos como la comunicación HTTP o el motor Unity influyeran en las mediciones obtenidas.

Una decisión metodológica importante consistió en medir directamente los proveedores de inteligencia artificial mediante adaptadores específicos para cada servicio, en lugar de realizar las pruebas a través de los endpoints FastAPI utilizados por la aplicación principal. De esta forma, las métricas obtenidas reflejan el comportamiento real de cada proveedor y no el desempeño de la infraestructura de comunicación.

La arquitectura del benchmark fue organizada en tres ejecutores independientes:

- `run_llm_benchmark.py`
- `run_stt_benchmark.py`
- `run_tts_benchmark.py`

Cada ejecutor carga un conjunto controlado de muestras, invoca el proveedor correspondiente múltiples veces y almacena los resultados obtenidos para su posterior análisis. Los resultados se registran tanto en formato CSV como JSONL, permitiendo la reproducibilidad y análisis posterior de los datos recolectados.

Entorno Experimental

Las pruebas fueron ejecutadas utilizando una combinación de servicios cloud y modelos locales.

Servicios Cloud

LLM

- Gemini 3.1 Flash Lite
- Gemini 2.5 Flash
- Groq Llama 3.1 8B Instant
- Groq Qwen3 32B

STT

- ElevenLabs Scribe v2
- Gemini 3.5 Flash Audio
- Groq Whisper Large v3 Turbo

TTS

- Eleven Multilingual v2
- Eleven Flash v2.5
- Gemini 3.1 Flash TTS Preview

Modelos Locales

LLM

- Gemma 3 mediante Ollama

STT

- Whisper Base
- Moonshine

TTS

- Piper TTS
- Kokoro TTS

La configuración de cada proveedor se realizó mediante variables de entorno, permitiendo modificar modelos y credenciales sin alterar el código fuente del benchmark.

Dataset de Evaluación

Se utilizaron datasets controlados para cada categoría evaluada.

LLM

Las pruebas de lenguaje utilizaron prompts almacenados en archivos JSONL versionados dentro del repositorio. Todos los modelos recibieron exactamente las mismas instrucciones para garantizar la comparabilidad de los resultados.

STT

Las pruebas de reconocimiento de voz utilizaron el mismo archivo de audio de referencia (nora.wav) para todos los proveedores evaluados. Adicionalmente, se almacenó una transcripción de referencia utilizada para el cálculo automático del Word Error Rate (WER).

TTS

Las pruebas de síntesis de voz utilizaron un conjunto fijo de frases almacenadas en archivos JSONL, garantizando que cada motor sintetizara exactamente el mismo contenido textual.

Métricas Evaluadas

La evaluación se realizó utilizando seis dimensiones propuestas en el enunciado del proyecto:

1. Latencia
2. Precisión y Calidad
3. Costo y Escalabilidad
4. Privacidad y Gobernanza
5. Customización y Flexibilidad
6. Facilidad de Integración

Latencia

La latencia fue medida utilizando la función `time.perf_counter()` de Python, la cual permite realizar mediciones de alta precisión temporal.

En LLM se registró:

- Tiempo total de respuesta.
- Tiempo hasta el primer token (TTFT), cuando el proveedor lo soportaba.

En STT se registró:

- Tiempo total requerido para transcribir el archivo de audio.

En TTS se registró:

- Tiempo total necesario para generar el archivo de audio sintetizado.

Las pruebas actuales fueron ejecutadas con tres repeticiones consecutivas por modelo, utilizando el parámetro configurable BENCHMARK_REPETITIONS. La metodología fue diseñada para soportar cinco o más repeticiones por proveedor con el fin de reducir la variabilidad estadística.

Precisión y Calidad

Para los sistemas STT se utilizó Word Error Rate (WER), una métrica ampliamente utilizada en reconocimiento automático del habla que cuantifica la distancia entre la transcripción generada y una referencia humana.

$$\text{WER} = (S + D + I) / N$$

donde:

- S representa sustituciones.
- D representa eliminaciones.
- I representa inserciones.
- N representa el número total de palabras de referencia.

Esta métrica ha sido ampliamente utilizada en la literatura de reconocimiento de voz y constituye uno de los indicadores estándar para comparar sistemas STT.

Para los modelos LLM y TTS se definieron métricas cualitativas adicionales (coherence_score, instruction_following_score, intelligibility_score y naturalness_score), las cuales se mantienen como parte de futuras extensiones del benchmark.

Almacenamiento y Procesamiento de Resultados

Los resultados fueron almacenados utilizando dos niveles de agregación:

Resultados Raw

- Una fila por ejecución individual.
- Permiten analizar dispersión, estabilidad y comportamiento de cada corrida.

Resultados Summary

Una fila agregada por combinación:

- sample_id
- provider
- model

Estas tablas contienen promedios de latencia y métricas agregadas utilizadas posteriormente para la elaboración de gráficos y tablas comparativas del reporte.

Análisis Comparativo por Categoría

Large Language Models (LLM)

Tabla Comparativa LLM

Modelo	Latencia Empírica (AVG 5 corridas)	Precisión y Calidad	Costo y Escalabilidad	Privacidad y Gobernanza	Customización y Flexibilidad	Facilidad de Integración
Gemini 3.1 Flash Lite	6810.94ms	Alta calidad para tareas generales y conversaciones	Free Tier disponible. Pago por millón de tokens. Diseñado para cargas de alto volumen.	Datos procesados en infraestructura Google Cloud.	Soporta prompts complejos, contexto largo y multimodalidad.	SDK oficial para Python, REST API y documentación extensa.
Gemini 2.5 Flash	4989.12ms	Excelente seguimiento de instrucciones y razonamiento general.	Free Tier disponible. Pago por millón de tokens.	Datos gestionados bajo políticas de Google.	Alta capacidad de personalización mediante system prompts.	Integración sencilla mediante SDK oficial.
Llama 3.1 8B Instant (Groq)	499.92ms	Muy buen desempeño para agentes conversacionales.	Free Tier disponible. Costos bajos por token.	Procesamiento cloud. Opciones empresariales de retención reducida.	Compatible con prompts avanzados y salidas estructuradas.	API simple y bien documentada.
Qwen3 32B (Groq)	1207.64ms	Alta capacidad para tareas complejas y razonamiento.	Free Tier disponible. Pago por consumo.	Procesamiento cloud mediante Groq.	Soporte para prompts sofisticados y formatos estructurados.	Integración similar al resto del ecosistema Groq.
Gemma 3 (Ollama)	5845.78ms	Calidad dependiente del hardware disponible.	Sin costo por token. Requiere recursos locales.	Máxima privacidad al ejecutarse localmente.	Control completo sobre el entorno de ejecución.	Integración local mediante Ollama API.

Análisis

Los resultados experimentales evidenciaron una diferencia sustancial entre las alternativas evaluadas. Los modelos ejecutados mediante Groq presentaron las menores latencias observadas, mientras que los modelos Gemini mostraron tiempos de respuesta significativamente superiores dentro del entorno experimental utilizado.

Gráfico de costos

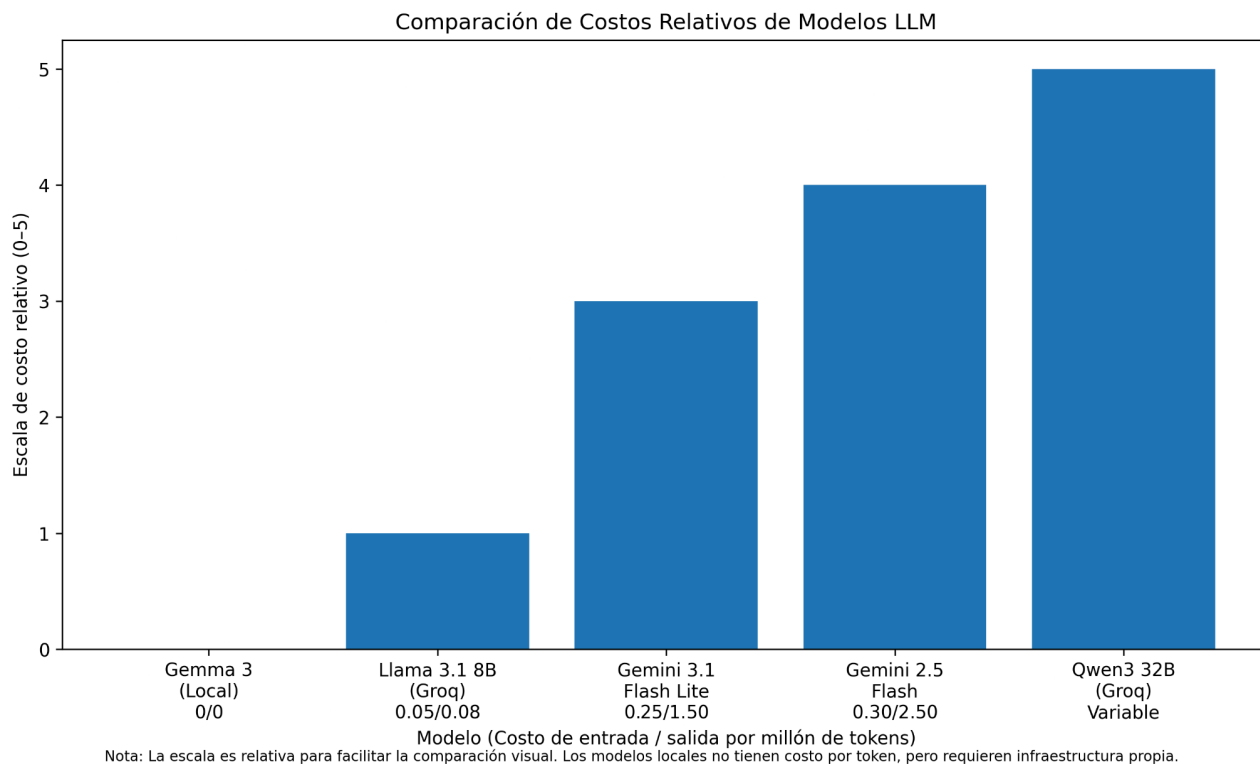
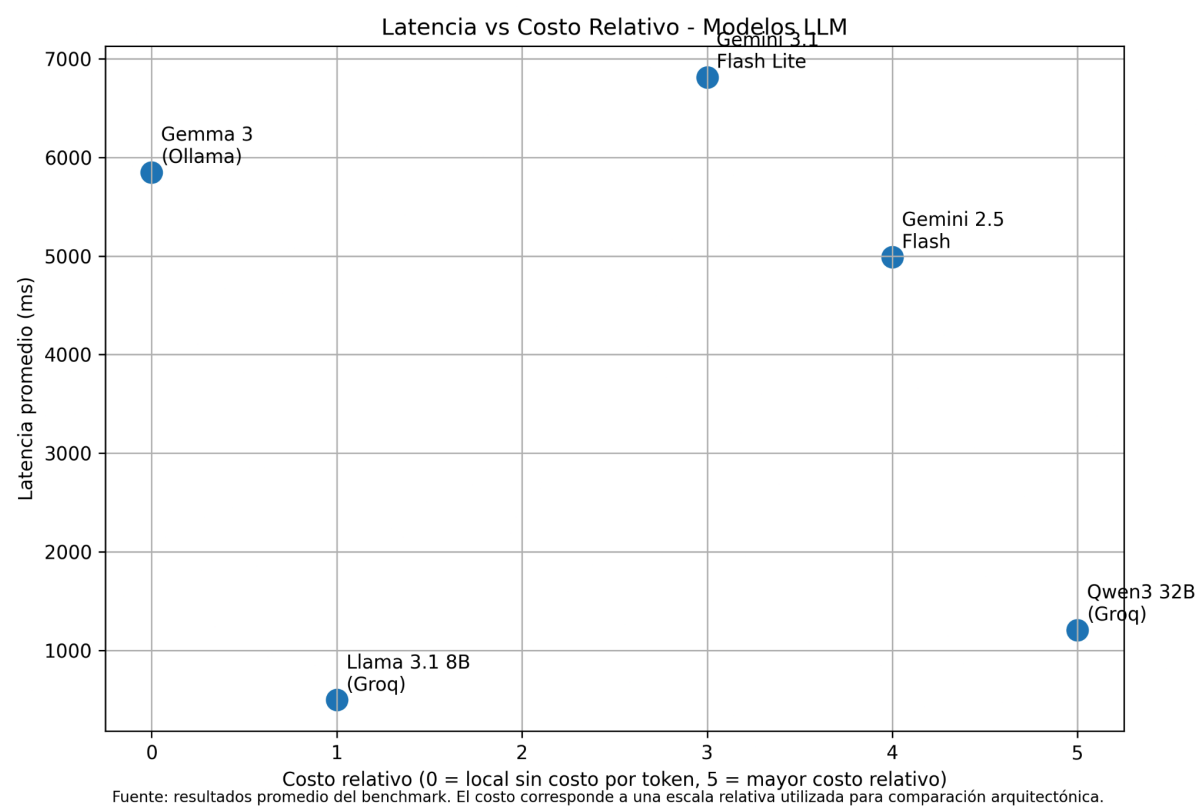


Gráfico latencia vs costo



Speech-to-Text (STT)

Tabla Comparativa STT

Modelo	Latencia Empírica (AVG 5 corridas)	Precisión y Calidad	Costo y Escalabilidad	Privacidad y Gobernanza	Customización y Flexibilidad	Facilidad de Integración
Scribe v2	2061.68ms	Diseñado para transcripción de alta calidad y producción.	Free Tier limitado mediante créditos. Pago por uso.	Procesamiento cloud mediante ElevenLabs.	Integración con ecosistema de voz ElevenLabs.	SDK y API bien documentadas.
Whisper Large v3 Turbo (Groq)	1924.96ms	Modelo basado en Whisper optimizado para velocidad.	Free Tier disponible. Pago por hora de audio.	Procesamiento cloud mediante Groq.	Limitada personalización de vocabulario.	Integración sencilla mediante API.

Whisper Base Local	1341.77ms	Precisión evaluada mediante WER.	Sin costo por uso. Requiere recursos locales.	Procesamiento completamente local.	Posibilidad de adaptar flujos de pre y post procesamiento .	Fácil integración con Python.
Gemini 3.5 flash	2747.81ms	Aprovecha capacidades multimodales para transcripción.	Free Tier disponible. Pago por consumo.	Procesamiento cloud mediante Google.	Integración directa con otros servicios Gemini.	SDK unificado con el ecosistema Gemini.
Moonshine	9987.32ms	Alternativa ligera orientada a ejecución local.	Sin costo por uso.	Procesamiento completamente local.	Menor madurez que Whisper, pero altamente portable.	Integración local mediante Python.

Análisis

Whisper Base Local obtuvo simultáneamente la menor latencia promedio y el menor Word Error Rate (WER) dentro del conjunto evaluado, posicionándose como la alternativa con mejor relación entre precisión, costo y privacidad. Sin embargo, en cuanto a naturalidad de la voz, el premio se lo siguen llevando los modelos de ElevenLabs.

Gráfico de costos

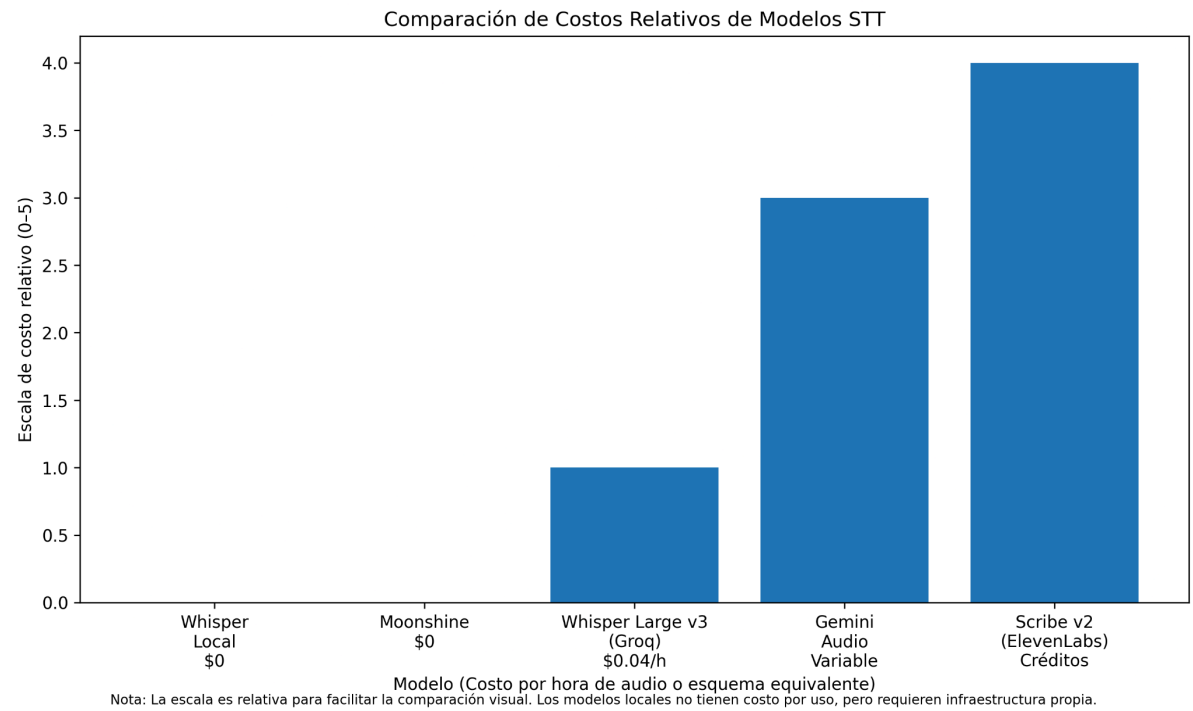
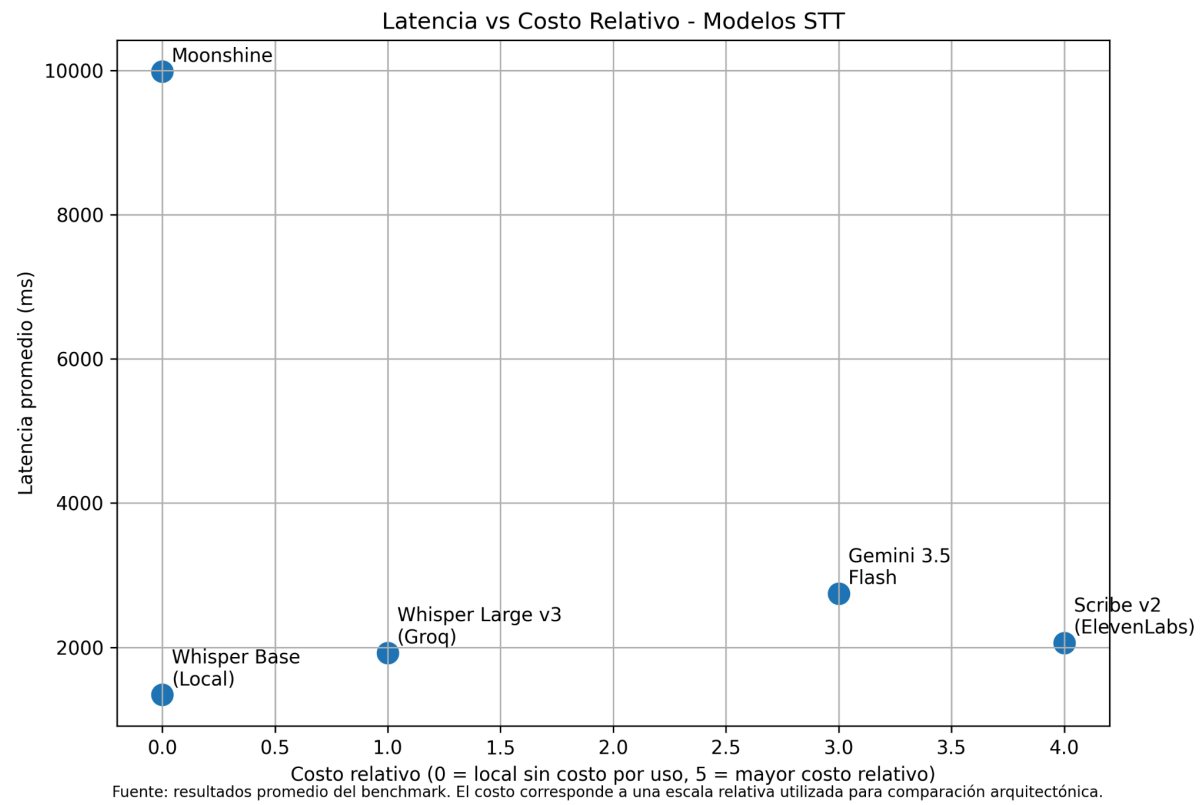


Gráfico latencia vs costo



Text-to-Speech (TTS)

Tabla Comparativa TTS

Modelo	Latencia Empírica (AVG 5 corridas)	Precisión y Calidad	Costo y Escalabilidad	Privacidad y Gobernanza	Customización y Flexibilidad	Facilidad de Integración
Eleven Multilingual v2	1308.80ms	Voz natural y expresiva.	Free Tier limitado mediante créditos. Pago por uso.	Procesamiento cloud mediante ElevenLabs.	Amplias capacidades de personalización y clonación de voz.	API madura y ampliamente utilizada.
Eleven Flash v2.5	528.59ms	Optimizado para generación rápida de audio.	Free Tier limitado mediante créditos.	Procesamiento cloud.	Soporte para múltiples configuraciones de voz.	Integración sencilla.

Gemini 3.1 Flash TTS	12798.31ms	Integración nativa con ecosistema Gemini.	Free Tier disponible. Pago por consumo.	Procesamiento cloud mediante Google.	Configuración mediante API y parámetros de voz soportados.	SDK unificado con Gemini.
Piper TTS	2303.55ms	Calidad adecuada para aplicaciones offline.	Sin costo por uso.	Procesamiento completamente local.	Personalización limitada a las voces disponibles.	Implementación simple y ligera.
Kokoro TTS	3135.62ms	Modelo moderno orientado a ejecución eficiente.	Sin costo por uso.	Procesamiento completamente local.	Soporte para múltiples voces según implementación.	Integración local mediante Python y ONNX.

Análisis

Las soluciones cloud ofrecen una experiencia más madura y mayor naturalidad percibida, mientras que Piper y Kokoro destacan por permitir síntesis local sin costos por inferencia. Kokoro resulta particularmente atractivo para proyectos académicos y prototipos debido a su facilidad de despliegue y bajo requerimiento computacional.

Gráfico de costos

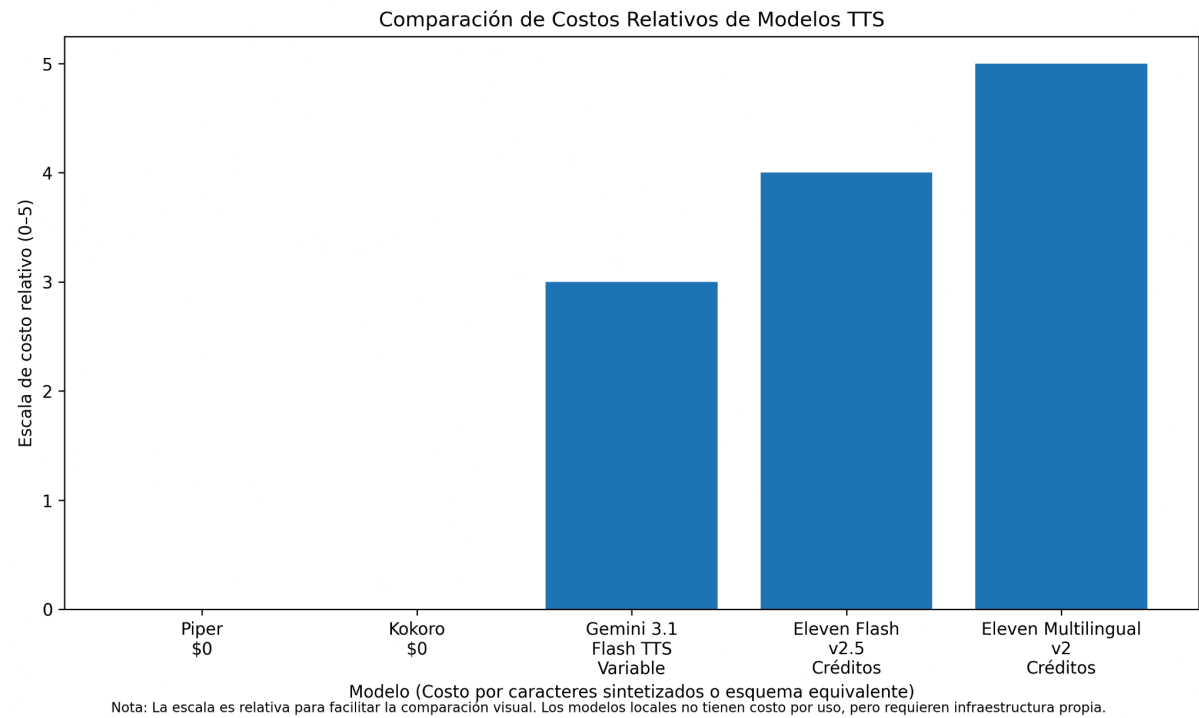
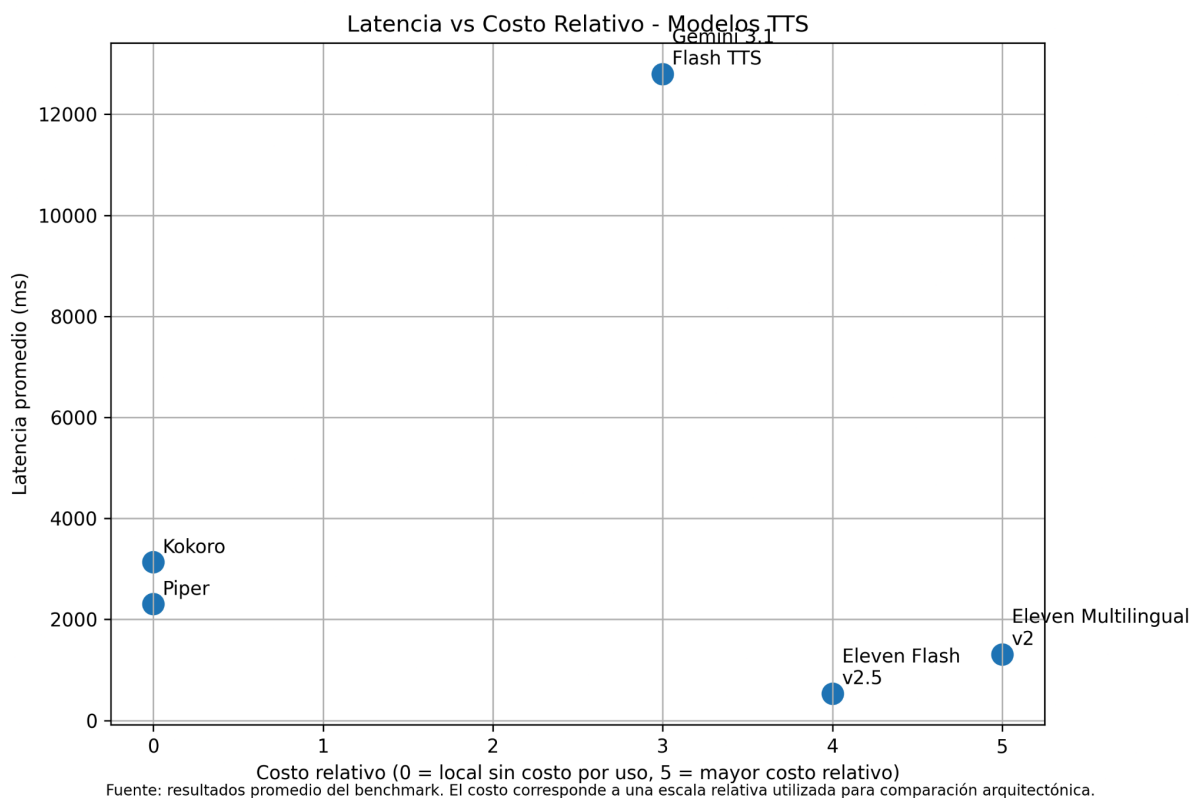


Gráfico latencia vs costo



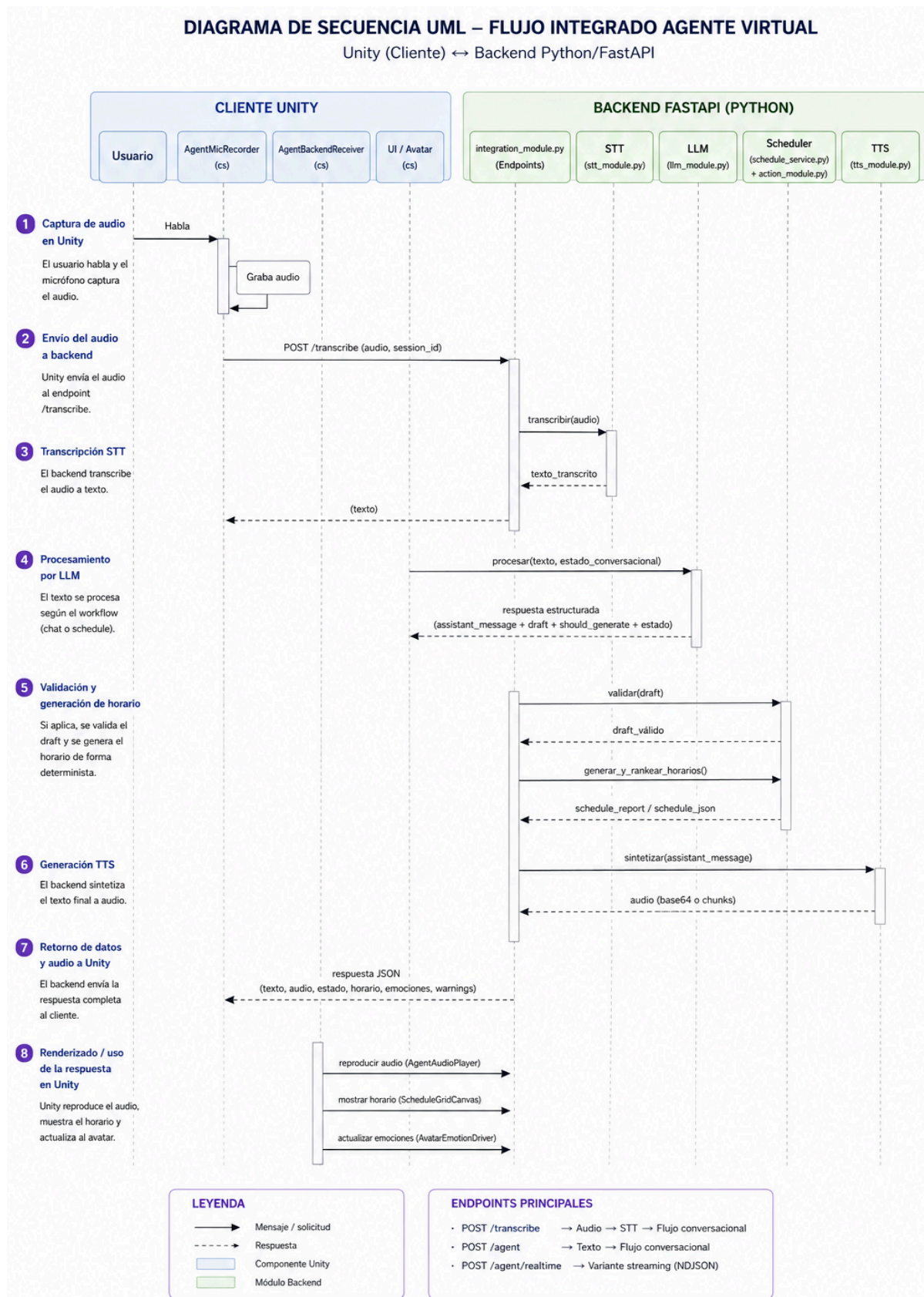
Síntesis General

El análisis evidencia una tendencia consistente entre las tres categorías evaluadas. Los servicios cloud ofrecen ventajas en facilidad de integración, escalabilidad y experiencia de usuario, mientras que las soluciones locales proporcionan mayor control, privacidad y ausencia de costos variables por uso.

En consecuencia, no existe una combinación universalmente óptima. La selección de tecnologías debe realizarse considerando los requisitos específicos de cada aplicación, especialmente en términos de latencia, privacidad, presupuesto y facilidad de mantenimiento. Los resultados de latencia empírica, TTFT y WER obtenidos mediante el benchmark complementan este análisis y permiten fundamentar cuantitativamente la selección arquitectónica final.

Arquitectura y Pipeline de Comunicación

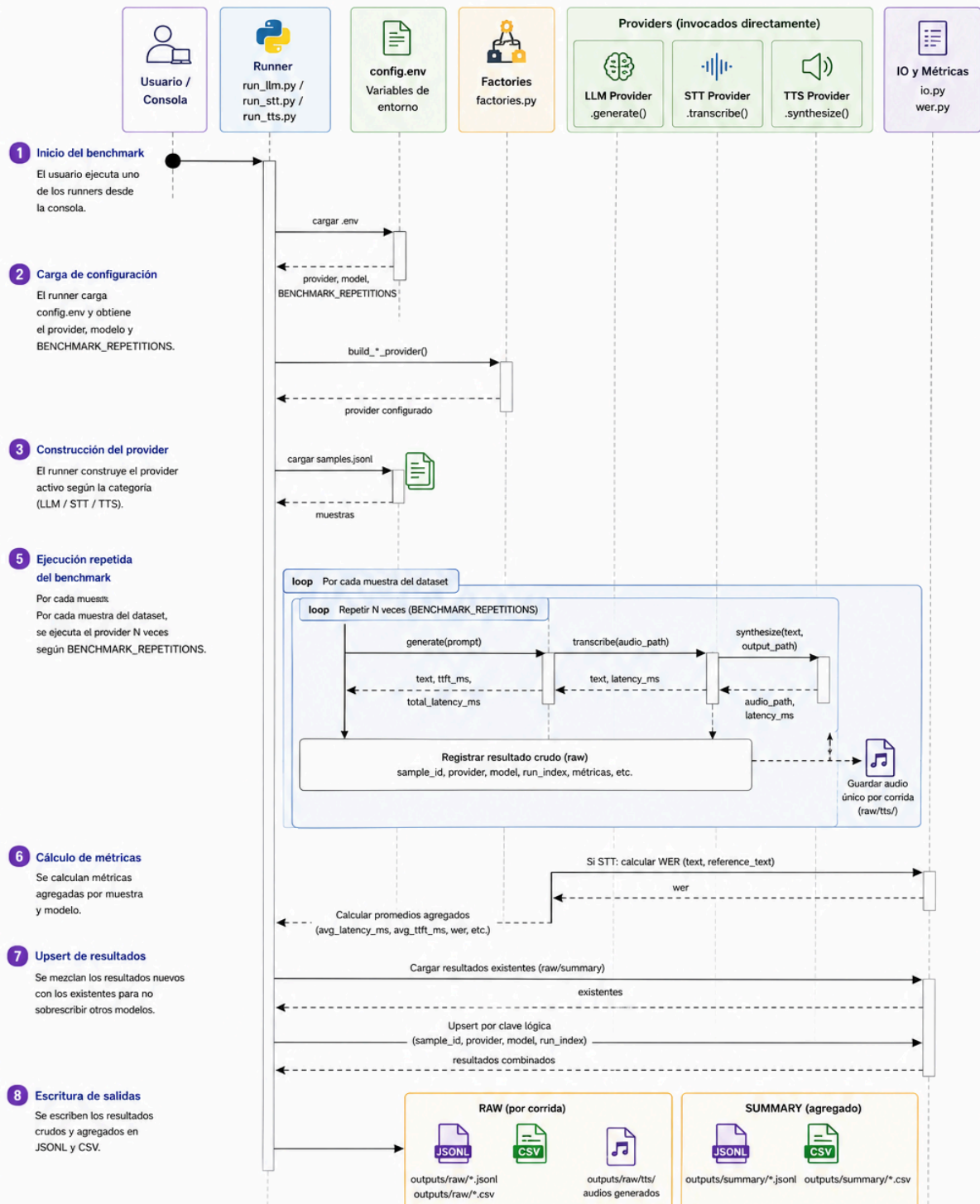
Arquitectura del sistema simplificada



Arquitectura del benchmarking

DIAGRAMA DE SECUENCIA UML – SISTEMA DE BENCHMARKS (LLM, STT, TTS)

Flujo de ejecución de los benchmarks desde la consola hasta la generación de resultados



Recomendaciones según Contexto y Conclusión

Escenario 1: Privacidad estricta y presupuesto cero

Para entornos donde la privacidad sea prioritaria y se desee evitar costos recurrentes, se recomienda una arquitectura completamente local:

- LLM: Gemma 3 (Ollama)
- STT: Whisper Local
- TTS: Kokoro o Piper

Esta configuración evita la transferencia de datos hacia servicios externos y permite operar incluso sin conexión a Internet. Aunque el desempeño puede ser inferior al de algunas alternativas cloud, ofrece el mayor nivel de control sobre la información procesada.

Escenario 2: Aplicaciones interactivas de baja latencia

Para sistemas conversacionales donde la rapidez de respuesta sea el principal requisito, se recomienda:

- LLM: Llama 3.1 8B Instant (Groq)
- STT: Whisper Base Local
- TTS: Eleven Flash v2.5

Esta combinación prioriza tiempos de respuesta reducidos y resulta adecuada para asistentes virtuales en tiempo real, experiencias inmersivas y aplicaciones con interacción continua.

Escenario 3: Experiencia de usuario y facilidad de integración

Para proyectos que prioricen calidad conversacional, facilidad de desarrollo y una experiencia de usuario consistente, se recomienda:

- LLM: Gemini Flash
- STT: ElevenLabs Scribe v2
- TTS: Eleven Multilingual v2

Esta arquitectura ofrece una integración sencilla con servicios modernos de IA y una experiencia de voz de alta calidad, siendo apropiada para demostraciones, prototipos avanzados y aplicaciones orientadas al usuario final.

Escenario 4: Arquitectura híbrida recomendada

Para el proyecto desarrollado, se recomienda una arquitectura híbrida basada en componentes intercambiables:

- LLM principal: Gemini Flash
- LLM alternativo: Llama 3.1 8B Instant (Groq)

- STT principal: Scribe v2
- STT alternativo: Whisper Local
- TTS principal: Eleven Multilingual v2
- TTS alternativo: Kokoro

Este enfoque permite equilibrar calidad, flexibilidad, privacidad y costos, facilitando además la sustitución de proveedores sin modificar la lógica principal de la aplicación.

Conclusión

Los resultados del benchmark evidencian que no existe una única combinación óptima para todos los escenarios. La selección de servicios debe realizarse considerando los objetivos específicos de cada aplicación, incluyendo requisitos de latencia, privacidad, costo y experiencia de usuario.

Los servicios cloud mostraron ventajas en integración y capacidad conversacional, mientras que las alternativas locales destacaron por su independencia de proveedores externos y ausencia de costos operativos. Asimismo, los resultados sugieren que una arquitectura modular basada en providers intercambiables constituye la estrategia más robusta, ya que permite adaptar el sistema a distintos contextos sin introducir cambios significativos en la arquitectura general.

En consecuencia, se recomienda mantener una separación clara entre los módulos LLM, STT y TTS, permitiendo seleccionar dinámicamente la combinación más adecuada según las necesidades operativas de cada despliegue.

Referencias

1. Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., ... & Amodei, D. (2020). *Language Models are Few-Shot Learners*. Advances in Neural Information Processing Systems, 33, 1877–1901.
2. Liang, P., Bommasani, R., Lee, T., Tsipras, D., Soylu, D., Yasunaga, M., ... & Liang, P. (2022). *Holistic Evaluation of Language Models (HELM)*. Transactions on Machine Learning Research.
3. Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., ... & Sutskever, I. (2023). *Robust Speech Recognition via Large-Scale Weak Supervision*. Proceedings of ICML 2023. (Whisper).
4. Google DeepMind. (2024). *Gemini Technical Report*.
5. Google DeepMind. (2024). *Gemma Technical Report*.
6. Meta AI. (2024). *The Llama 3 Herd of Models*.
7. ElevenLabs. (2025). *ElevenLabs API Documentation: Speech-to-Text and Text-to-Speech*.
8. Groq. (2025). *Groq Documentation and Inference Platform Documentation*.
9. Ollama. (2025). *Ollama Documentation*.
10. OpenAI. (2023). *Whisper Documentation and Model Overview*.